ADESH PATANKAR

Title: Implement an application for counting the number of words in a text file using Pig.

Outcome: Students will be able to implement an application for counting the number of words in a text file using Pig.

Theory:

Apache Pig

Apache Pig is an abstraction over MapReduce. It is a tool/platform which is used to analyze larger sets of data representing them as data flows. Pig is generally used with **Hadoop**; we can perform all the data manipulation operations in Hadoop using Apache Pig.

To write data analysis programs, Pig provides a high-level language known as **Pig Latin**. This language provides various operators using which programmers can develop their own functions for reading, writing, and processing data.

To analyze data using **Apache Pig**, programmers need to write scripts using Pig Latin language. All these scripts are internally converted to Map and Reduce tasks. Apache Pig has a component known as **Pig Engine** that accepts the Pig Latin scripts as input and converts those scripts into MapReduce jobs.

Why Do We Need Apache Pig?

Programmers who are not so good at Java normally used to struggle working with Hadoop, especially while performing any MapReduce tasks. Apache Pig is a boon for all such programmers.

- Using **Pig Latin**, programmers can perform MapReduce tasks easily without having to type complex codes in Java.
- Apache Pig uses **multi-query approach**, thereby reducing the length of codes. For example, an operation that would require you to type 200 lines of code (LoC) in Java can be easily done by typing as less as just 10 LoC in Apache Pig. Ultimately Apache Pig reduces the development time by almost 16 times.
- Pig Latin is **SQL-like language** and it is easy to learn Apache Pig when you are familiar with SQL.
- Apache Pig provides many built-in operators to support data operations like joins, filters, ordering, etc. In addition, it also provides nested data types like tuples, bags, and maps that are missing from MapReduce.

Features of Pig

Apache Pig comes with the following features −

- **Rich set of operators** − It provides many operators to perform operations like join, sort, filer, etc.
- **Ease of programming** − Pig Latin is similar to SQL and it is easy to write a Pig script if you are good at SQL.
- **Optimization opportunities** − The tasks in Apache Pig optimize their execution automatically, so the programmers need to focus only on semantics of the language.
- **Extensibility** − Using the existing operators, users can develop their own functions to read, process, and write data.
- **UDF's** − Pig provides the facility to create **User-defined Functions** in other programming languages such as Java and invoke or embed them in Pig Scripts. • **Handles all kinds of data** − Apache Pig analyzes all kinds of data, both structured as well as unstructured. It stores the results in HDFS.

Apache Pig Vs MapReduce

Listed below are the major differences between Apache Pig and MapReduce.

Apache Pig	MapReduce
Apache Pig is a data flow language.	MapReduce is a data processing paradigm.
It is a high level language.	MapReduce is low level and rigid.
Performing a Join operation in Apache Pig is pretty simple.	It is quite difficult in MapReduce to perform a Join operation between datasets.
Any novice programmer with a basic knowledge of SQL can work conveniently with Apache Pig.	Exposure to Java is must to work with MapReduce.
Apache Pig uses multi-query approach, thereby reducing the length of the codes to a great extent.	MapReduce will require almost 20 times more the number of lines to perform the same task.
There is no need for compilation. On execution, every Apache Pig operator is converted internally into a MapReduce job.	MapReduce jobs have a long compilation process.

Applications of Apache Pig

Apache Pig is generally used by data scientists for performing tasks involving ad-hoc processing and quick prototyping. Apache Pig is used −

- To process huge data sources such as web logs.

- To perform data processing for search platforms.
- To process time sensitive data loads.

Apache Pig – History

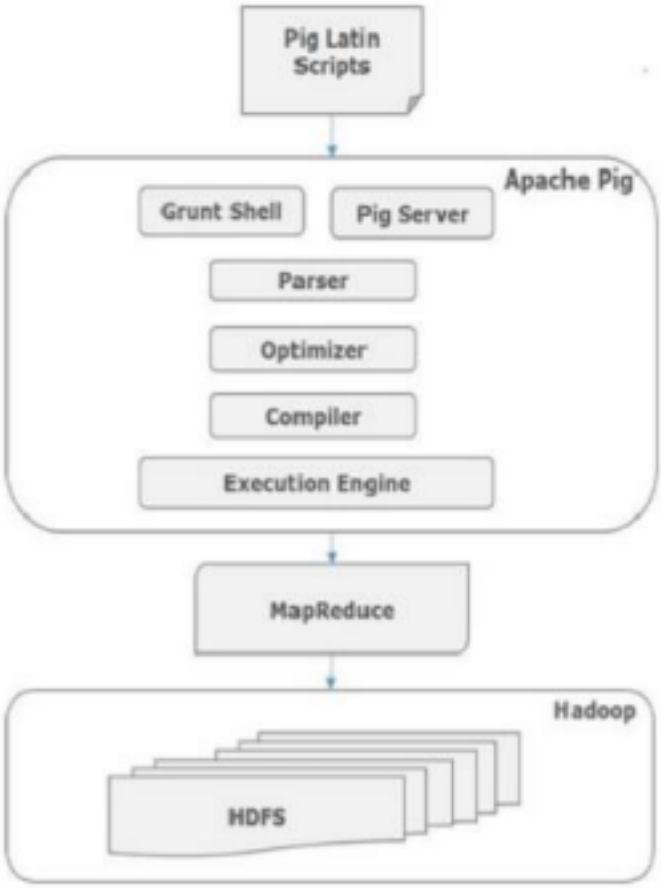
In **2006**, Apache Pig was developed as a research project at Yahoo, especially to create and execute MapReduce jobs on every dataset. In **2007**, Apache Pig was open sourced via Apache incubator. In **2008**, the first release of Apache Pig came out. In **2010**, Apache Pig graduated as an Apache top-level project.

Apache Pig - Architecture

The language used to analyze data in Hadoop using Pig is known as **Pig Latin**. It is a highlevel data processing language which provides a rich set of data types and operators to perform various operations on the data.

To perform a particular task Programmers using Pig, programmers need to write a Pig script using the Pig Latin language, and execute them using any of the execution mechanisms (Grunt Shell, UDFs, Embedded). After execution, these scripts will go through a series of transformations applied by the Pig Framework, to produce the desired output.

Internally, Apache Pig converts these scripts into a series of MapReduce jobs, and thus, it makes the programmer's job easy. The architecture of Apache Pig is shown below.



Apache Pig Components

As shown in the figure, there are various components in the Apache Pig framework. Let us take a look at the major components.

Parser

Initially the Pig Scripts are handled by the Parser. It checks the syntax of the script, does type checking, and other miscellaneous checks. The output of the parser will be a DAG (directed acyclic graph), which represents the Pig Latin statements and logical operators.
In the DAG, the logical operators of the script are represented as the nodes and the data flows are represented as edges.

Optimizer

The logical plan (DAG) is passed to the logical optimizer, which carries out the logical optimizations such as projection and pushdown.

Compiler

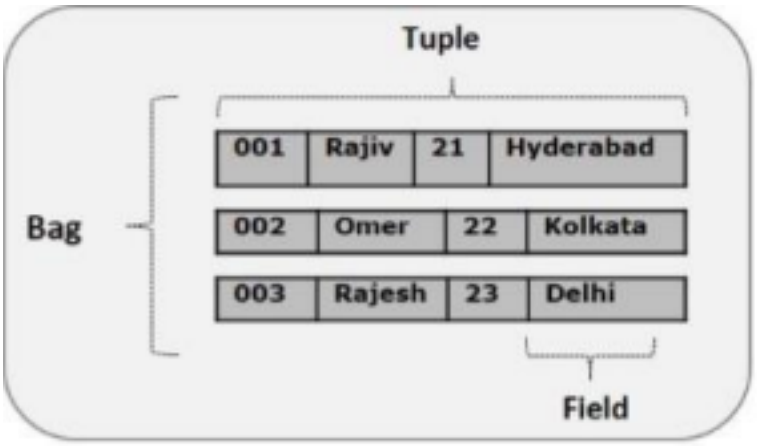
The compiler compiles the optimized logical plan into a series of MapReduce

jobs. **Execution engine**

Finally the MapReduce jobs are submitted to Hadoop in a sorted order. Finally, these MapReduce jobs are executed on Hadoop producing the desired results.

Pig Latin Data Model

The data model of Pig Latin is fully nested and it allows complex non-atomic datatypes such as **map** and **tuple**. Given below is the diagrammatical representation of Pig Latin's data model.



Atom

Any single value in Pig Latin, irrespective of their data, type is known as an **Atom**. It is stored as string and can be used as string and number. int, long, float, double, chararray, and bytearray are the atomic values of Pig. A piece of data or a simple atomic value is known as a **field**.

Example − 'raja' or '30'

Tuple

A record that is formed by an ordered set of fields is known as a tuple, the fields can be of any type. A tuple is similar to a row in a table of RDBMS.

Example − (Raja, 30)

Bag

A bag is an unordered set of tuples. In other words, a collection of tuples (non-unique) is known as a bag. Each tuple can have any number of fields (flexible schema). A bag is represented by '{}'. It is similar to a table in RDBMS, but unlike a table in RDBMS, it is not necessary that every tuple contain the same number of fields or that the fields in the same position (column) have the same type.
Example − {(Raja, 30), (Mohammad, 45)}

A bag can be a field in a relation; in that context, it is known as **inner**

bag. **Example** − {Raja, 30, **{9848022338, raja@gmail.com,}**}

Map

A map (or data map) is a set of key-value pairs. The **key** needs to be of type chararray and should be unique. The **value** might be of any type. It is represented by '[]'

Example − [name#Raja, age#30]

Relation

A relation is a bag of tuples. The relations in Pig Latin are unordered (there is no guarantee that tuples are processed in any particular order).

Apache Pig - Installation

Prerequisites

It is essential that you have Hadoop and Java installed on your system before you go for Apache Pig. Therefore, prior to installing Apache Pig, install Hadoop and Java by following the steps given in the following link −

http://www.tutorialspoint.com/hadoop/hadoop_enviornment_setup.htm

Download Apache Pig

First of all, download the latest version of Apache Pig from the following website − https://pig.apache.org/

Step 1

Open the homepage of Apache Pig website. Under the section **News,** click on the link **release page** as shown in the following snapshot.



Step 2

On clicking the specified link, you will be redirected to the **Apache Pig Releases** page. On this page, under the **Download** section, you will have two links, namely, **Pig 0.8 and later** and **Pig 0.7 and before**. Click on the link **Pig 0.8 and later**, then you will be redirected to the page having a set of mirrors.



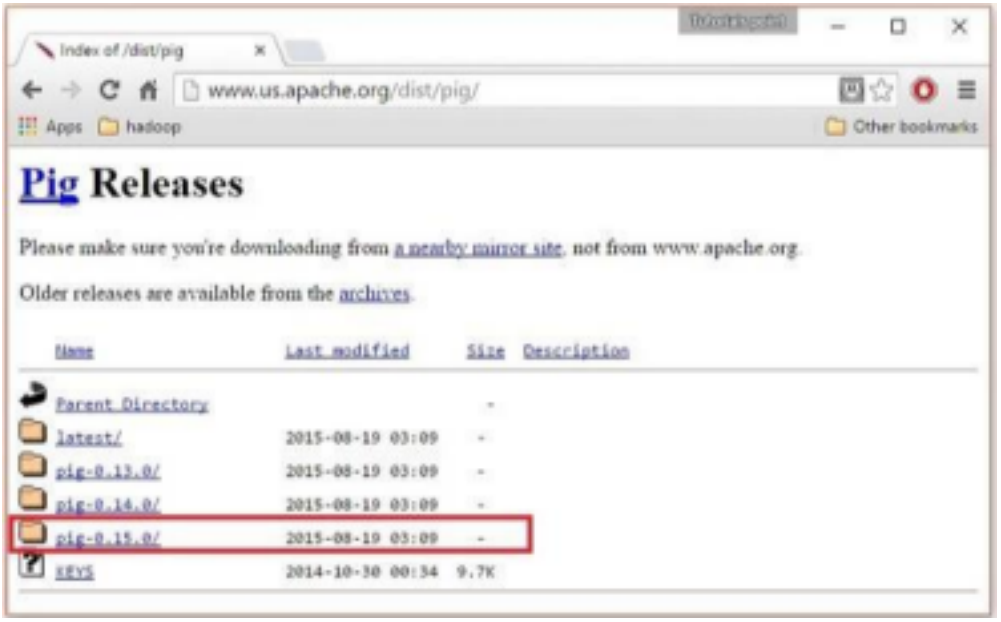
Step 3

Choose and click any one of these mirrors as shown below.



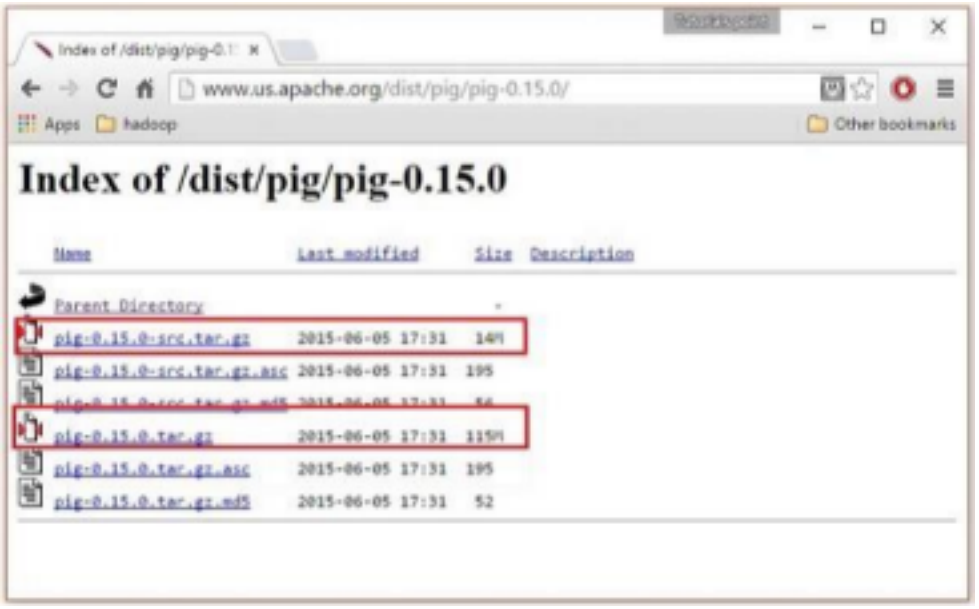
Step 4

These mirrors will take you to the **Pig Releases** page. This page contains various versions of Apache Pig. Click the latest version among them.



Step 5

Within these folders, you will have the source and binary files of Apache Pig in various distributions. Download the tar files of the source and binary files of Apache Pig 0.15, **pig0.15.0-src.tar.gz** and **pig-0.15.0.tar.gz.**



Install Apache Pig

After downloading the Apache Pig software, install it in your Linux environment by following the steps given below.

Step 1

Create a directory with the name Pig in the same directory where the installation directories of **Hadoop, Java,** and other software were installed. (In our tutorial, we have created the Pig directory in the user named Hadoop).

```
$ mkdir Pig
```

Step 2

Extract the downloaded tar files as shown below.

```
$ cd Downloads/
$ tar zxvf pig-0.15.0-src.tar.gz
$ tar zxvf pig-0.15.0.tar.gz
```

Step 3

Move the content of **pig-0.15.0-src.tar.gz** file to the **Pig** directory created earlier as shown below.

```
$ mv pig-0.15.0-src.tar.gz/* /home/Hadoop/Pig/
```

Configure Apache Pig

After installing Apache Pig, we have to configure it. To configure, we need to edit two files − **bashrc and pig.properties**.

.bashrc file

In the **.bashrc** file, set the following variables −

- **PIG_HOME** folder to the Apache Pig's installation folder,
- **PATH** environment variable to the bin folder, and
- **PIG_CLASSPATH** environment variable to the etc (configuration) folder of your Hadoop installations (the directory that contains the core-site.xml, hdfs site.xml and mapred-site.xml files).

```
export PIG_HOME = /home/Hadoop/Pig
export PATH = $PATH:/home/Hadoop/pig/bin
export PIG_CLASSPATH = $HADOOP_HOME/conf
```

Verifying the Installation

Verify the installation of Apache Pig by typing the version command. If the installation is successful, you will get the version of Apache Pig as shown below.

```
$ pig –version
```

```
Apache Pig version 0.15.0 (r1682971)
compiled Jun 01 2015, 11:44:35
```

Apache Pig - Reading Data

In general, Apache Pig works on top of Hadoop. It is an analytical tool that analyzes large datasets that exist in the **H**adoop **F**ile **S**ystem. To analyze data using Apache Pig, we have to initially load the data into Apache Pig. This chapter explains how to load data to Apache Pig from HDFS.

Preparing HDFS

In MapReduce mode, Pig reads (loads) data from HDFS and stores the results back in HDFS. Therefore, let us start HDFS and create the following sample data in HDFS.

Student ID	First Name	Last Name	Phone	City
001	Rajiv	Reddy	9848022337	Hyderabad
002	siddarth	Battacharya	9848022338	Kolkata
003	Rajesh	Khanna	9848022339	Delhi
004	Preethi	Agarwal	9848022330	Pune

005	Trupthi	Mohanthy	9848022336	Bhuwaneshwar
006	Archana	Mishra	9848022335	Chennai

The above dataset contains personal details like id, first name, last name, phone number and city, of six students.

Step 1: Verifying Hadoop

First of all, verify the installation using Hadoop version command, as shown below.

```
$ hadoop version
```

If your system contains Hadoop, and if you have set the PATH variable, then you will get the following output −

```
Hadoop 2.6.0
Subversion https://git-wip-us.apache.org/repos/asf/hadoop.git -r
e3496499ecb8d220fba99dc5ed4c99c8f9e33bb1
Compiled by jenkins on 2014-11-13T21:10Z
Compiled with protoc 2.5.0
From source with checksum 18e43357c8f927c0695f1e9522859d6a
This command was run using
/home/Hadoop/hadoop/share/hadoop/common/hadoop common-2.6.0.jar
```

Step 2: Starting HDFS

Browse through the **sbin** directory of Hadoop and start **yarn** and Hadoop dfs (distributed file system) as shown below.

```
cd /$Hadoop_Home/sbin/
$ start-dfs.sh
localhost: starting namenode, logging to /home/Hadoop/hadoop/logs/hadoopHadoop
namenode-localhost.localdomain.out
localhost: starting datanode, logging to /home/Hadoop/hadoop/logs/hadoopHadoop-datanode
localhost.localdomain.out
Starting secondary namenodes [0.0.0.0]
starting secondarynamenode, logging to /home/Hadoop/hadoop/logs/hadoop
Hadoopsecondarynamenode-localhost.localdomain.out

$ start-yarn.sh
starting yarn daemons
starting resourcemanager, logging to /home/Hadoop/hadoop/logs/yarn
Hadoopresourcemanager-localhost.localdomain.out
localhost: starting nodemanager, logging to /home/Hadoop/hadoop/logs/yarnHadoop
nodemanager-localhost.localdomain.out
```

Step 3: Create a Directory in HDFS

In Hadoop DFS, you can create directories using the command **mkdir**. Create a new directory in HDFS with the name **Pig_Data** in the required path as shown below.

```
$cd /$Hadoop_Home/bin/
$ hdfs dfs -mkdir hdfs://localhost:9000/Pig_Data
```

Step 4: Placing the data in HDFS

The input file of Pig contains each tuple/record in individual lines. And the entities of the record are separated by a delimiter (In our example we used **“,”**).

In the local file system, create an input file **student_data.txt** containing data as shown below.

```
001,Rajiv,Reddy,9848022337,Hyderabad
002,siddarth,Battacharya,9848022338,Kolkata
003,Rajesh,Khanna,9848022339,Delhi
004,Preethi,Agarwal,9848022330,Pune
005,Trupthi,Mohanthy,9848022336,Bhuwaneshwar
006,Archana,Mishra,9848022335,Chennai.
```

Now, move the file from the local file system to HDFS using **put** command as shown below. (You can use **copyFromLocal** command as well.)

```
$ cd $HADOOP_HOME/bin
$ hdfs dfs -put /home/Hadoop/Pig/Pig_Data/student_data.txt dfs://localhost:9000/pig_data/
```

Verifying the file

You can use the **cat** command to verify whether the file has been moved into the HDFS, as shown below.

```
$ cd $HADOOP_HOME/bin
$ hdfs dfs -cat hdfs://localhost:9000/pig_data/student_data.txt
```

Output

You can see the content of the file as shown below.

```
15/10/01 12:16:55 WARN util.NativeCodeLoader: Unable to load native-hadoop
library for your platform... using builtin-java classes where applicable

001,Rajiv,Reddy,9848022337,Hyderabad
002,siddarth,Battacharya,9848022338,Kolkata
003,Rajesh,Khanna,9848022339,Delhi
004,Preethi,Agarwal,9848022330,Pune
005,Trupthi,Mohanthy,9848022336,Bhuwaneshwar
006,Archana,Mishra,9848022335,Chennai
```

The Load Operator

You can load data into Apache Pig from the file system (HDFS/ Local) using **LOAD** operator of **Pig Latin**.

Syntax

The load statement consists of two parts divided by the "=" operator. On the left-hand side, we need to mention the name of the relation **where** we want to store the data, and on the right hand side, we have to define **how** we store the data. Given below is the syntax of the **Load** operator.

```
Relation_name = LOAD 'Input file path' USING function as schema;
```

Where,

- **relation_name** − We have to mention the relation in which we want to store the data.
- **Input file path** − We have to mention the HDFS directory where the file is stored. (In MapReduce mode)
- **function** − We have to choose a function from the set of load functions provided by Apache Pig (**BinStorage, JsonLoader, PigStorage, TextLoader**).
- **Schema** − We have to define the schema of the data. We can define the required schema as follows −

```
(column1 : data type, column2 : data type, column3 : data type);
```

Note − We load the data without specifying the schema. In that case, the columns will be addressed as $01, $02, etc… (check).

Example

As an example, let us load the data in **student_data.txt** in Pig under the schema named **Student** using the **LOAD** command.

Start the Pig Grunt Shell

First of all, open the Linux terminal. Start the Pig Grunt shell in MapReduce mode as shown below.

```
$ Pig –x mapreduce
```

It will start the Pig Grunt shell as shown below.

```
15/10/01 12:33:37 INFO pig.ExecTypeProvider: Trying ExecType : LOCAL 15/10/01
12:33:37 INFO pig.ExecTypeProvider: Trying ExecType : MAPREDUCE 15/10/01
12:33:37 INFO pig.ExecTypeProvider: Picked MAPREDUCE as the ExecType
2015-10-01 12:33:38,080 [main] INFO org.apache.pig.Main - Apache Pig version 0.15.0
(r1682971) compiled Jun 01 2015, 11:44:35
2015-10-01 12:33:38,080 [main] INFO org.apache.pig.Main - Logging error messages to:
/home/Hadoop/pig_1443683018078.log
2015-10-01 12:33:38,242 [main] INFO org.apache.pig.impl.util.Utils - Default bootup file
/home/Hadoop/.pigbootup not found
```

2015-10-01 12:33:39,630 [main]
INFO org.apache.pig.backend.hadoop.executionengine.HExecutionEngine - Connecting to hadoop file system at: hdfs://localhost:9000

grunt>

Execute the Load Statement

Now load the data from the file **student_data.txt** into Pig by executing the following Pig Latin statement in the Grunt shell.

```
grunt> student = LOAD
'hdfs://localhost:9000/pig_data/student_data.txt' USING
PigStorage(',')
 as ( id:int, firstname:chararray, lastname:chararray, phone:chararray,
 city:chararray );
```

Simple Word Count using Apache Pig

The basic hello world program in hadoop is the word count program. I have explained the word count implementation using java mapreduce and hive queries in my previous posts. Here I am explaining the implementation of basic word count logic using pig script. TOKENIZE is a build in function available in apache pig which tokenizes a line into words. Similarly FLATTEN and COUNT are also built-in functions available in apache pig. You can check the output or flow of each step by using DUMP command after every step. The below pig scripts will do the count of words in the input file.

```
A = load 'data.txt' as (line:chararray);

B = foreach A generate TOKENIZE(line) as tokens;

C = foreach B generate flatten(tokens) as words;

D = group C by words;

E = foreach D generate group, COUNT(C);

F = order E by $1;
dump F;
```

Commands:-

```
a = load '/pig/Ex7Input'
```

b = foreach a generate flatten(TOKENIZE((chararray)$0)) as word;

c = group b by word;

d = foreach c generate COUNT(b), group;

dump d

Output:-----

HadoopVersion PigVersion UserId StartedAt FinishedAt Features 3.3.4 0.17.0 hadoop

2023-07-07 17:45:36 2023-07-07 17:48:07 GROUP_BY

Success!

Job Stats (time in seconds):

JobId Maps Reduces MaxMapTime MinMapTime AvgMapTime MedianMapTime MaxReduceTime MinReduceTime AvgReduceTime MedianReducetime Alias Feature Outputs

job_1672229227626_0001 1 1 n/a n/a n/a n/a n/a n/a n/a a,b,c,d GROUP_BY,COMBINER hdfs://0.0.0.0:9000/tmp/temp 64966117/tmp-908099740,

Input(s):

Successfully read 0 records from: "/pig/Ex7Input"
Output(s):

Successfully stored 0 records in: "hdfs://0.0.0.0:9000/tmp/temp-64966117/tmp-908099740"

Counters:

Total records written : 0

Total bytes written : 0

Spillable Memory Manager spill count : 0

Total bags proactively spilled: 0

Total records proactively spilled: 0

Job DAG:

job_1672229227626_0001

(1,a)

(1,as)

(1,to)

(1,and)

(1,go.)

(1,had)

(1,its)

(1,the)

(2,was)

(2,Mary)

(2,lamb)

(1,snow)

(1,sure)

(1,that)

(1,went)
(1,white)

(1,fleece)

(1,little)

(1,everywhere).

Results :

```
EXPERIMENT 7 : Counting the number of words in a text file using Apache Pig
Part 1 : Environment check, input file and upload to HDFS

$ pig -version
Apache Pig version 0.18.0 (r1926484)
compiled Jun 16 2025, 10:38:21
$ bash -c hadoop version | head -1
Hadoop 3.3.6
$ jps
1696 NameNode
3377 JobHistoryServer
2483 ResourceManager
37816 Jps
1916 DataNode
2157 SecondaryNameNode
2687 NodeManager
$ cat data/input.txt
Big Data is a collection of data that is huge in volume
Hadoop stores big data in HDFS and processes it with MapReduce
Apache Pig is a platform for analysing big data on Hadoop
Pig Latin scripts are converted into MapReduce jobs by Pig
Pig makes big data processing simple and fast
$ cat data/q4.txt
Big Data is useful. Big Data is powerful.
$ hdfs dfs -mkdir -p /bdt/exp7
$ hdfs dfs -put -f data/input.txt data/q4.txt /bdt/exp7/
$ hdfs dfs -ls /bdt/exp7
Found 2 items
-rw-r--r--   1 ubuntu supergroup        282 2026-10-25 20:04 /bdt/exp7/input.txt
-rw-r--r--   1 ubuntu supergroup         42 2026-10-25 20:04 /bdt/exp7/q4.txt
```

```
EXPERIMENT 7 : Counting the number of words in a text file using Apache Pig
Part 2 : LOAD the text file and display its contents (DESCRIBE + DUMP)

$ cat pig/01_load_display.pig
-- Experiment 7 | Part 2 : LOAD a text file and display its contents
-- TextLoader reads every line of the file as one chararray field
lines = LOAD '/bdt/exp7/input.txt' USING TextLoader() AS (line:chararray);
DESCRIBE lines;
DUMP lines;

$ pig -x mapreduce -4 /home/ubuntu/bdt/pig/log4j.properties -f pig/01_load_display.pig
2026-10-25 20:07:25,011 INFO pig.ExecTypeProvider: Trying ExecType : LOCAL
2026-10-25 20:07:25,012 INFO pig.ExecTypeProvider: Trying ExecType : MAPREDUCE
2026-10-25 20:07:25,012 INFO pig.ExecTypeProvider: Picked MAPREDUCE as the ExecType
2026-10-25 20:07:25,039 INFO pig.Main: Loaded log4j properties from file: /home/ubuntu/bdt/pig/log4j.properties
20:07:25 [INFO] Main - Apache Pig version 0.18.0 (r1926484) compiled Jun 16 2025, 10:38:21
20:07:25 [INFO] Main - Logging error messages to: /home/ubuntu/bdt/exp7/pig_1792936045037.log
20:07:25 [INFO] HExecutionEngine - Connecting to hadoop file system at: hdfs://localhost:9000
20:07:25 [WARN] PigServer - ATS is disabled since yarn.timeline-service.enabled set to false
lines: {line: chararray}
20:07:27 [INFO] MapReduceLauncher - 1 map-reduce job(s) waiting for submission.
20:07:27 [WARN] JobResourceUploader - No job jar file set.  User classes may not be found. See Job or Job#setJar(String).
20:07:28 [INFO] MapReduceLauncher - HadoopJobId: job_1792936583670_0061
20:07:28 [INFO] MapReduceLauncher - Processing aliases lines
20:07:28 [INFO] MapReduceLauncher - detailed locations: M: lines[3,8],lines[-1,-1] C:  R:
20:07:28 [INFO] MapReduceLauncher - 0% complete
20:07:28 [INFO] MapReduceLauncher - Running jobs are [job_1792936583670_0061]
20:07:36 [INFO] MapReduceLauncher - 50% complete
20:07:36 [INFO] MapReduceLauncher - Running jobs are [job_1792936583670_0061]
20:07:39 [INFO] MapReduceLauncher - 100% complete
20:07:39 [INFO] SimplePigStats - Script Statistics:

HadoopVersion   PigVersion      UserId  StartedAt       FinishedAt      Features
3.3.6   0.18.0  ubuntu  2026-10-25 20:07:26     2026-10-25 20:07:39     UNKNOWN

Success!

Job Stats (time in seconds):
JobId   Maps    Reduces MaxMapTime      MinMapTime      AvgMapTime      MedianMapTime   MaxReduceTime   MinReduceTime   AvgReduceTime   MedianReducetime        Alias   Feature Outputs
job_1792936583670_0061  1       0       1       1       1       1       0       0       0       0       lines   MAP_ONLY        hdfs://localhost:9000/tmp/temp2097005113/tmp-1496808966,

Input(s):
Successfully read 5 records (649 bytes) from: "/bdt/exp7/input.txt"

Output(s):
Successfully stored 5 records (307 bytes) in: "hdfs://localhost:9000/tmp/temp2097005113/tmp-1496808966"

Counters:
Total records written : 5
Total bytes written : 307
Spillable Memory Manager spill count : 0
Total bags proactively spilled: 0
Total records proactively spilled: 0

Job DAG:
job_1792936583670_0061


20:07:39 [INFO] MapReduceLauncher - Success!
(Big Data is a collection of data that is huge in volume)
(Hadoop stores big data in HDFS and processes it with MapReduce)
(Apache Pig is a platform for analysing big data on Hadoop)
(Pig Latin scripts are converted into MapReduce jobs by Pig)
(Pig makes big data processing simple and fast)
20:07:39 [INFO] Main - Pig script completed in 14 seconds and 877 milliseconds (14877 ms)
```

```
EXPERIMENT 7 : Counting the number of words in a text file using Apache Pig
Part 3 : Word frequency: TOKENIZE -> FLATTEN -> GROUP -> COUNT -> ORDER -> STORE

$ cat pig/02_word_count.pig
-- Experiment 7 | Part 3 : count the frequency of every distinct word
lines   = LOAD '/bdt/exp7/input.txt' AS (line:chararray);
tokens  = FOREACH lines  GENERATE TOKENIZE(line) AS tokens;   -- bag of words per line
words   = FOREACH tokens GENERATE FLATTEN(tokens) AS word;    -- one tuple per word
grouped = GROUP words BY word;                                -- all occurrences of a word together
counts  = FOREACH grouped GENERATE group AS word, COUNT(words) AS freq;
sorted  = ORDER counts BY freq DESC, word ASC;
DESCRIBE words;
DESCRIBE grouped;
DESCRIBE counts;
-- write the result to HDFS and print it
rmf /bdt/exp7/output_wordcount
STORE sorted INTO '/bdt/exp7/output_wordcount' USING PigStorage('\t');
fs -ls /bdt/exp7/output_wordcount
fs -cat /bdt/exp7/output_wordcount/part*

$ pig -x mapreduce -4 /home/ubuntu/bdt/pig/log4j.properties -f pig/02_word_count.pig
2026-10-25 20:09:35,409 INFO pig.ExecTypeProvider: Trying ExecType : LOCAL
2026-10-25 20:09:35,409 INFO pig.ExecTypeProvider: Trying ExecType : MAPREDUCE
2026-10-25 20:09:35,409 INFO pig.ExecTypeProvider: Picked MAPREDUCE as the ExecType
2026-10-25 20:09:35,435 INFO pig.Main: Loaded log4j properties from file: /home/ubuntu/bdt/pig/log4j.properties
20:09:35 [INFO] Main - Apache Pig version 0.18.0 (r1924484) compiled Jun 16 2025, 10:38:21
20:09:35 [INFO] Main - Logging error messages to: /home/ubuntu/bdt/exp7/pig_1792939175433.log
20:09:35 [INFO] HExecutionEngine - Connecting to hadoop file system at: hdfs://localhost:9000
20:09:35 [WARN] PigServer - ATS is disabled since yarn.timeline-service.enabled set to false
words: {word: chararray}
grouped: {group: chararray,words: {(word: chararray)}}
counts: {word: chararray,freq: long}
20:09:37 [INFO] MapReduceLauncher - 1 map-reduce job(s) waiting for submission.
20:09:37 [WARN] JobResourceUploader - No job jar file set.  User classes may not be found. See Job or Job#setJar(String).
20:09:38 [INFO] MapReduceLauncher - HadoopJobId: job_1792936583670_0062
20:09:38 [INFO] MapReduceLauncher - Processing aliases counts,grouped,lines,tokens,words
20:09:38 [INFO] MapReduceLauncher - detailed locations: M: lines[2,10],tokens[-1,-1],words[4,10],counts[6,10],grouped[5,10] C: counts[6,10],grouped[5,10] R: counts[6,10]
20:09:38 [INFO] MapReduceLauncher - 0% complete
20:09:38 [INFO] MapReduceLauncher - Running jobs are [job_1792936583670_0062]
20:09:46 [INFO] MapReduceLauncher - 16% complete
20:09:46 [INFO] MapReduceLauncher - Running jobs are [job_1792936583670_0062]
20:09:51 [INFO] MapReduceLauncher - 33% complete
20:09:51 [INFO] MapReduceLauncher - Running jobs are [job_1792936583670_0062]
20:09:55 [INFO] MapReduceLauncher - 1 map-reduce job(s) waiting for submission.
20:09:55 [WARN] JobResourceUploader - No job jar file set.  User classes may not be found. See Job or Job#setJar(String).
20:09:56 [INFO] MapReduceLauncher - HadoopJobId: job_1792936583670_0063
20:09:56 [INFO] MapReduceLauncher - Processing aliases sorted
20:09:56 [INFO] MapReduceLauncher - detailed locations: M: sorted[7,10] C:  R:
20:10:05 [INFO] MapReduceLauncher - 50% complete
20:10:05 [INFO] MapReduceLauncher - Running jobs are [job_1792936583670_0063]
20:10:10 [INFO] MapReduceLauncher - 66% complete
20:10:10 [INFO] MapReduceLauncher - Running jobs are [job_1792936583670_0063]
20:10:12 [INFO] MapReduceLauncher - 1 map-reduce job(s) waiting for submission.
20:10:12 [WARN] JobResourceUploader - No job jar file set.  User classes may not be found. See Job or Job#setJar(String).
20:10:13 [INFO] MapReduceLauncher - HadoopJobId: job_1792936583670_0064
20:10:13 [INFO] MapReduceLauncher - Processing aliases sorted
20:10:13 [INFO] MapReduceLauncher - detailed locations: M: sorted[7,10] C:  R:
20:10:22 [INFO] MapReduceLauncher - 83% complete
20:10:22 [INFO] MapReduceLauncher - Running jobs are [job_1792936583670_0064]
20:10:27 [INFO] MapReduceLauncher - Running jobs are [job_1792936583670_0064]
20:10:33 [INFO] MapReduceLauncher - 100% complete
20:10:33 [INFO] SimplePigStats - Script Statistics:

HadoopVersion   PigVersion      UserId  StartedAt       FinishedAt      Features
3.3.6   0.18.0  ubuntu  2026-10-25 20:09:36     2026-10-25 20:10:33     GROUP_BY,ORDER_BY

Success!

Job Stats (time in seconds):
JobId   Maps    Reduces MaxMapTime      MinMapTime      AvgMapTime      MedianMapTime   MaxReduceTime   MinReduceTime   AvgReduceTime   MedianReducetime        Alias   Feature Outputs
job_1792936583670_0062  1       1       1       1       1       2       2       2       2       counts,grouped,lines,tokens,words       GROUP_BY,COMBINER
job_1792936583670_0063  1       1       1       1       1       1       1       1       1       sorted  SAMPLER
job_1792936583670_0064  1       1       1       1       1       1       1       1       1       sorted  ORDER_BY        /bdt/exp7/output_wordcount,

Input(s):
```

```
HadoopVersion   PigVersion      UserId  StartedAt       FinishedAt      Features
3.3.6   0.18.0  ubuntu  2026-10-25 20:09:36     2026-10-25 20:10:33     GROUP_BY,ORDER_BY

Success!

Job Stats (time in seconds):
JobId   Maps    Reduces MaxMapTime      MinMapTime      AvgMapTime      MedianMapTime   MaxReduceTime   MinReduceTime   AvgReduceTime   MedianReducetime        Alias   Feature Outputs
job_1792936583670_0062  1       1       1       1       1       2       2       2       2       counts,grouped,lines,tokens,words       GROUP_BY,COMBINER
job_1792936583670_0063  1       1       1       1       1       1       1       1       1       sorted  SAMPLER
job_1792936583670_0064  1       1       1       1       1       1       1       1       1       sorted  ORDER_BY        /bdt/exp7/output_wordcount,

Input(s):
Successfully read 5 records (649 bytes) from: "/bdt/exp7/input.txt"

Output(s):
Successfully stored 37 records (289 bytes) in: "/bdt/exp7/output_wordcount"

Counters:
Total records written : 37
Total bytes written : 289
Spillable Memory Manager spill count : 0
Total bags proactively spilled: 0
Total records proactively spilled: 0

Job DAG:
job_1792936583670_0062  ->      job_1792936583670_0063,
job_1792936583670_0063  ->      job_1792936583670_0064,
job_1792936583670_0064


20:10:33 [INFO] MapReduceLauncher - Success!
Found 2 items
-rw-r--r--   1 ubuntu supergroup          0 2026-10-25 20:10 /bdt/exp7/output_wordcount/_SUCCESS
-rw-r--r--   1 ubuntu supergroup        289 2026-10-25 20:10 /bdt/exp7/output_wordcount/part-r-00000
Pig     4
data    4
big     3
is      3
Hadoop  2
MapReduce       2
a       2
and     2
in      2
Apache  1
Big     1
Data    1
HDFS    1
Latin   1
analysing       1
are     1
by      1
collection      1
converted       1
fast    1
for     1
huge    1
into    1
it      1
jobs    1
makes   1
of      1
on      1
platform        1
processes       1
processing      1
scripts 1
simple  1
stores  1
that    1
volume  1
with    1
20:10:34 [INFO] Main - Pig script completed in 58 seconds and 708 milliseconds (58708 ms)

[ screenshot 3 - take it, then press Enter ]
```

```
EXPERIMENT 7 : Counting the number of words in a text file using Apache Pig
Part 4 : Total number of words, distinct words and lines

$ cat pig/03_total_words.pig
-- Experiment 7 | Part 4 : total number of words (and lines) in the file
lines     = LOAD '/bdt/exp7/input.txt' AS (line:chararray);
words     = FOREACH lines GENERATE FLATTEN(TOKENIZE(line)) AS word;
all_words = GROUP words ALL;
all_lines = GROUP lines ALL;
total     = FOREACH all_words GENERATE COUNT(words) AS total_words;
distinct_words = DISTINCT words;
all_distinct   = GROUP distinct_words ALL;
n_distinct     = FOREACH all_distinct GENERATE COUNT(distinct_words) AS distinct_words;
n_lines        = FOREACH all_lines GENERATE COUNT(lines) AS total_lines;
DUMP total;
DUMP n_distinct;
DUMP n_lines;

$ pig -x mapreduce -4 /home/ubuntu/bdt/pig/log4j.properties -f pig/03_total_words.pig
2026-10-25 20:10:49,265 INFO pig.ExecTypeProvider: Trying ExecType : LOCAL
2026-10-25 20:10:49,265 INFO pig.ExecTypeProvider: Trying ExecType : MAPREDUCE
2026-10-25 20:10:49,265 INFO pig.ExecTypeProvider: Picked MAPREDUCE as the ExecType
2026-10-25 20:10:49,295 INFO pig.Main: Loaded log4j properties from file: /home/ubuntu/bdt/pig/log4j.properties
20:10:49 [INFO] Main - Apache Pig version 0.18.0 (r1926484) compiled Jun 16 2025, 10:38:21
20:10:49 [INFO] Main - Logging error messages to: /home/ubuntu/bdt/exp7/pig_1792939249293.log
20:10:49 [INFO] HExecutionEngine - Connecting to hadoop file system at: hdfs://localhost:9000
20:10:49 [WARN] PigServer - ATS is disabled since yarn.timeline-service.enabled set to false
20:10:51 [INFO] MapReduceLauncher - 1 map-reduce job(s) waiting for submission.
20:10:51 [WARN] JobResourceUploader - No job jar file set.  User classes may not be found. See Job or Job#setJar(String).
20:10:52 [INFO] MapReduceLauncher - HadoopJobId: job_1792936583670_0065
20:10:52 [INFO] MapReduceLauncher - Processing aliases all_words,lines,total,words
20:10:52 [INFO] MapReduceLauncher - detailed locations: M: lines[2,12],words[-1,-1],total[6,12],all_words[4,12] C: total[6,12],all_words[4,12] R: total[6,12]
20:10:52 [INFO] MapReduceLauncher - 0% complete
20:10:52 [INFO] MapReduceLauncher - Running jobs are [job_1792936583670_0065]
20:10:59 [INFO] MapReduceLauncher - 50% complete
20:10:59 [INFO] MapReduceLauncher - Running jobs are [job_1792936583670_0065]
20:11:02 [INFO] MapReduceLauncher - Running jobs are [job_1792936583670_0065]
20:11:07 [INFO] MapReduceLauncher - 100% complete
20:11:07 [INFO] SimplePigStats - Script Statistics:

HadoopVersion   PigVersion      UserId  StartedAt       FinishedAt      Features
3.3.6   0.18.0  ubuntu  2026-10-25 20:10:50     2026-10-25 20:11:07     GROUP_BY

Success!

Job Stats (time in seconds):
JobId   Maps    Reduces MaxMapTime      MinMapTime      AvgMapTime      MedianMapTime   MaxReduceTime   MinReduceTime   AvgReduceTime   MedianReducetime        Alias   Feature Outputs
job_1792936583670_0065  1       1       1       1       1       1       1       1       all_words,lines,total,words     GROUP_BY,COMBINER       hdfs://localhost:9000/tmp/temp-914432951/tmp1651645224,

Input(s):
Successfully read 5 records (649 bytes) from: "/bdt/exp7/input.txt"

Output(s):
Successfully stored 1 records (13 bytes) in: "hdfs://localhost:9000/tmp/temp-914432951/tmp1651645224"

Counters:
Total records written : 1
Total bytes written : 13
Spillable Memory Manager spill count : 0
Total bags proactively spilled: 0
Total records proactively spilled: 0

Job DAG:
job_1792936583670_0065


20:11:07 [INFO] MapReduceLauncher - Success!
(52)
20:11:08 [INFO] MapReduceLauncher - 1 map-reduce job(s) waiting for submission.
20:11:08 [WARN] JobResourceUploader - No job jar file set.  User classes may not be found. See Job or Job#setJar(String).
20:11:09 [INFO] MapReduceLauncher - HadoopJobId: job_1792936583670_0066
20:11:09 [INFO] MapReduceLauncher - Processing aliases lines,words
20:11:09 [INFO] MapReduceLauncher - detailed locations: M: lines[2,12],words[-1,-1] C:  R:
20:11:09 [INFO] MapReduceLauncher - 0% complete
20:11:09 [INFO] MapReduceLauncher - Running jobs are [job_1792936583670_0066]
```

```
HadoopVersion   PigVersion      UserId  StartedAt       FinishedAt      Features
3.3.6   0.18.0  ubuntu  2026-10-25 20:11:07     2026-10-25 20:11:41     GROUP_BY,DISTINCT

Success!

Job Stats (time in seconds):
JobId   Maps    Reduces MaxMapTime      MinMapTime      AvgMapTime      MedianMapTime   MaxReduceTime   MinReduceTime   AvgReduceTime   MedianReducetime        Alias   Feature Outputs
job_1792936583670_0066  1       1       1       1       1       1       2       2       2       lines,words     DISTINCT
job_1792936583670_0067  1       1       1       1       1       1       1       1       1       all_distinct,n_distinct GROUP_BY,COMBINER       hdfs://localhost:9000/tmp/temp-914432951/tmp-1588667776,

Input(s):
Successfully read 5 records (649 bytes) from: "/bdt/exp7/input.txt"

Output(s):
Successfully stored 1 records (13 bytes) in: "hdfs://localhost:9000/tmp/temp-914432951/tmp-1588667776"

Counters:
Total records written : 1
Total bytes written : 13
Spillable Memory Manager spill count : 0
Total bags proactively spilled: 0
Total records proactively spilled: 0

Job DAG:
job_1792936583670_0066  ->      job_1792936583670_0067,
job_1792936583670_0067


20:11:41 [INFO] MapReduceLauncher - Success!
(37)
20:11:44 [INFO] MapReduceLauncher - 1 map-reduce job(s) waiting for submission.
20:11:44 [WARN] JobResourceUploader - No job jar file set.  User classes may not be found. See Job or Job#setJar(String).
20:11:45 [INFO] MapReduceLauncher - HadoopJobId: job_1792936583670_0068
20:11:45 [INFO] MapReduceLauncher - Processing aliases all_lines,lines,n_lines
20:11:45 [INFO] MapReduceLauncher - detailed locations: M: lines[2,12],lines[-1,-1],n_lines[10,17],all_lines[5,12] C: n_lines[10,17],all_lines[5,12] R: n_lines[10,17]
20:11:45 [INFO] MapReduceLauncher - 0% complete
20:11:45 [INFO] MapReduceLauncher - Running jobs are [job_1792936583670_0068]
20:11:52 [INFO] MapReduceLauncher - 50% complete
20:11:52 [INFO] MapReduceLauncher - Running jobs are [job_1792936583670_0068]
20:11:57 [INFO] MapReduceLauncher - Running jobs are [job_1792936583670_0068]
20:12:01 [INFO] MapReduceLauncher - 100% complete
20:12:01 [INFO] SimplePigStats - Script Statistics:

HadoopVersion   PigVersion      UserId  StartedAt       FinishedAt      Features
3.3.6   0.18.0  ubuntu  2026-10-25 20:11:41     2026-10-25 20:12:01     GROUP_BY

Success!

Job Stats (time in seconds):
JobId   Maps    Reduces MaxMapTime      MinMapTime      AvgMapTime      MedianMapTime   MaxReduceTime   MinReduceTime   AvgReduceTime   MedianReducetime        Alias   Feature Outputs
job_1792936583670_0068  1       1       1       1       1       1       1       1       1       all_lines,lines,n_lines GROUP_BY,COMBINER       hdfs://localhost:9000/tmp/temp-914432951/tmp-2139928166,

Input(s):
Successfully read 5 records (649 bytes) from: "/bdt/exp7/input.txt"

Output(s):
Successfully stored 1 records (13 bytes) in: "hdfs://localhost:9000/tmp/temp-914432951/tmp-2139928166"

Counters:
Total records written : 1
Total bytes written : 13
Spillable Memory Manager spill count : 0
Total bags proactively spilled: 0
Total records proactively spilled: 0

Job DAG:
job_1792936583670_0068


20:12:01 [INFO] MapReduceLauncher - Success!
(5)
20:12:01 [INFO] Main - Pig script completed in 1 minute, 12 seconds and 54 milliseconds (72054 ms)

[ screenshot 4 - take it, then press Enter ]
```

```
EXPERIMENT 7 : Counting the number of words in a text file using Apache Pig
Part 5 : Case sensitivity and punctuation: 'Big Data is useful. Big Data is powerful.'

$ cat pig/04_case_punctuation.pig
-- Experiment 7 | Part 5 : input "Big Data is useful. Big Data is powerful."
-- (a) plain TOKENIZE: case-sensitive and punctuation stays attached to words
raw = LOAD '/bdt/exp7/q4.txt' AS (line:chararray);
w1  = FOREACH raw GENERATE FLATTEN(TOKENIZE(line)) AS word;
g1  = GROUP w1 BY word;
c1  = FOREACH g1 GENERATE group AS word, COUNT(w1) AS freq;
DUMP c1;
-- (b) cleaned: punctuation removed and converted to lower case before counting
clean = FOREACH raw GENERATE LOWER(REPLACE(line, '[^a-zA-Z ]', '')) AS line;
w2  = FOREACH clean GENERATE FLATTEN(TOKENIZE(line)) AS word;
g2  = GROUP w2 BY word;
c2  = FOREACH g2 GENERATE group AS word, COUNT(w2) AS freq;
o2  = ORDER c2 BY freq DESC, word ASC;
DUMP o2;

$ pig -x mapreduce -4 /home/ubuntu/bdt/pig/log4j.properties -f pig/04_case_punctuation.pig
2026-10-25 20:12:19,489 INFO pig.ExecTypeProvider: Trying ExecType : LOCAL
2026-10-25 20:12:19,490 INFO pig.ExecTypeProvider: Trying ExecType : MAPREDUCE
2026-10-25 20:12:19,490 INFO pig.ExecTypeProvider: Picked MAPREDUCE as the ExecType
2026-10-25 20:12:19,519 INFO pig.Main: Loaded log4j properties from file: /home/ubuntu/bdt/pig/log4j.properties
20:12:19 [INFO] Main - Apache Pig version 0.18.0 (r1926484) compiled Jun 16 2025, 10:38:21
20:12:19 [INFO] Main - Logging error messages to: /home/ubuntu/bdt/exp7/pig_1792939339517.log
20:12:19 [INFO] HExecutionEngine - Connecting to hadoop file system at: hdfs://localhost:9000
20:12:19 [WARN] PigServer - ATS is disabled since yarn.timeline-service.enabled set to false
20:12:21 [INFO] MapReduceLauncher - 1 map-reduce job(s) waiting for submission.
20:12:21 [WARN] JobResourceUploader - No job jar file set.  User classes may not be found. See Job or Job#setJar(String).
20:12:22 [INFO] MapReduceLauncher - HadoopJobId: job_1792936583670_0069
20:12:22 [INFO] MapReduceLauncher - Processing aliases c1,g1,raw,w1
20:12:22 [INFO] MapReduceLauncher - detailed locations: M: raw[3,6],w1[-1,-1],c1[6,6],g1[5,6] C: c1[6,6],g1[5,6] R: c1[6,6]
20:12:22 [INFO] MapReduceLauncher - 0% complete
20:12:22 [INFO] MapReduceLauncher - Running jobs are [job_1792936583670_0069]
20:12:30 [INFO] MapReduceLauncher - 50% complete
20:12:30 [INFO] MapReduceLauncher - Running jobs are [job_1792936583670_0069]
20:12:33 [INFO] MapReduceLauncher - Running jobs are [job_1792936583670_0069]
20:12:38 [INFO] MapReduceLauncher - 100% complete
20:12:38 [INFO] SimplePigStats - Script Statistics:

HadoopVersion   PigVersion      UserId  StartedAt       FinishedAt      Features
3.3.6   0.18.0  ubuntu  2026-10-25 20:12:20     2026-10-25 20:12:38     GROUP_BY

Success!

Job Stats (time in seconds):
JobId   Maps    Reduces MaxMapTime      MinMapTime      AvgMapTime      MedianMapTime   MaxReduceTime   MinReduceTime   AvgReduceTime   MedianReducetime        Alias   Feature Outputs
job_1792936583670_0069  1       1       1       1       1       1       1       1       c1,g1,raw,w1    GROUP_BY,COMBINER       hdfs://localhost:9000/tmp/temp1833660981/tmp1249192299,

Input(s):
Successfully read 1 records (406 bytes) from: "/bdt/exp7/q4.txt"

Output(s):
Successfully stored 5 records (63 bytes) in: "hdfs://localhost:9000/tmp/temp1833660981/tmp1249192299"

Counters:
Total records written : 5
Total bytes written : 63
Spillable Memory Manager spill count : 0
Total bags proactively spilled: 0
Total records proactively spilled: 0

Job DAG:
job_1792936583670_0069


20:12:38 [INFO] MapReduceLauncher - Success!
(is,2)
(Big,2)
(Data,2)
(useful.,1)
(powerful.,1)
20:12:40 [INFO] MapReduceLauncher - 1 map-reduce job(s) waiting for submission.
20:12:40 [WARN] JobResourceUploader - No job jar file set.  User classes may not be found. See Job or Job#setJar(String).
```

```
(is,2)
(Big,2)
(Data,2)
(useful.,1)
(powerful.,1)
20:12:40 [INFO] MapReduceLauncher - 1 map-reduce job(s) waiting for submission.
20:12:40 [WARN] JobResourceUploader - No job jar file set.  User classes may not be found. See Job or Job#setJar(String).
20:12:41 [INFO] MapReduceLauncher - HadoopJobId: job_1792936583670_0070
20:12:41 [INFO] MapReduceLauncher - Processing aliases c2,clean,g2,raw,w2
20:12:41 [INFO] MapReduceLauncher - detailed locations: M: raw[3,6],clean[-1,-1],w2[10,6],c2[12,6],g2[11,6] C: c2[12,6],g2[11,6] R: c2[12,6]
20:12:41 [INFO] MapReduceLauncher - 0% complete
20:12:41 [INFO] MapReduceLauncher - Running jobs are [job_1792936583670_0070]
20:12:48 [INFO] MapReduceLauncher - 16% complete
20:12:48 [INFO] MapReduceLauncher - Running jobs are [job_1792936583670_0070]
20:12:53 [INFO] MapReduceLauncher - 33% complete
20:12:53 [INFO] MapReduceLauncher - Running jobs are [job_1792936583670_0070]
20:12:58 [INFO] MapReduceLauncher - 1 map-reduce job(s) waiting for submission.
20:12:58 [WARN] JobResourceUploader - No job jar file set.  User classes may not be found. See Job or Job#setJar(String).
20:12:59 [INFO] MapReduceLauncher - HadoopJobId: job_1792936583670_0071
20:12:59 [INFO] MapReduceLauncher - Processing aliases o2
20:12:59 [INFO] MapReduceLauncher - detailed locations: M: o2[13,6] C:  R:
20:13:06 [INFO] MapReduceLauncher - 50% complete
20:13:06 [INFO] MapReduceLauncher - Running jobs are [job_1792936583670_0071]
20:13:11 [INFO] MapReduceLauncher - 66% complete
20:13:11 [INFO] MapReduceLauncher - Running jobs are [job_1792936583670_0071]
20:13:14 [INFO] MapReduceLauncher - 1 map-reduce job(s) waiting for submission.
20:13:14 [WARN] JobResourceUploader - No job jar file set.  User classes may not be found. See Job or Job#setJar(String).
20:13:16 [INFO] MapReduceLauncher - HadoopJobId: job_1792936583670_0072
20:13:16 [INFO] MapReduceLauncher - Processing aliases o2
20:13:16 [INFO] MapReduceLauncher - detailed locations: M: o2[13,6] C:  R:
20:13:25 [INFO] MapReduceLauncher - 83% complete
20:13:25 [INFO] MapReduceLauncher - Running jobs are [job_1792936583670_0072]
20:13:28 [INFO] MapReduceLauncher - Running jobs are [job_1792936583670_0072]
20:13:32 [INFO] MapReduceLauncher - 100% complete
20:13:32 [INFO] SimplePigStats - Script Statistics:

HadoopVersion   PigVersion      UserId  StartedAt       FinishedAt      Features
3.3.6   0.18.0  ubuntu  2026-10-25 20:12:38     2026-10-25 20:13:32     GROUP_BY,ORDER_BY

Success!

Job Stats (time in seconds):
JobId   Maps    Reduces MaxMapTime      MinMapTime      AvgMapTime      MedianMapTime   MaxReduceTime   MinReduceTime   AvgReduceTime   MedianReducetime        Alias   Feature Outputs
job_1792936583670_0070  1       1       1       1       1       1       1       1       c2,clean,g2,raw,w2      GROUP_BY,COMBINER
job_1792936583670_0071  1       1       1       1       1       1       1       1       o2      SAMPLER
job_1792936583670_0072  1       1       1       1       1       1       1       1       o2      ORDER_BY        hdfs://localhost:9000/tmp/temp1833660981/tmp-1553344936,

Input(s):
Successfully read 1 records (406 bytes) from: "/bdt/exp7/q4.txt"

Output(s):
Successfully stored 5 records (61 bytes) in: "hdfs://localhost:9000/tmp/temp1833660981/tmp-1553344936"

Counters:
Total records written : 5
Total bytes written : 61
Spillable Memory Manager spill count : 0
Total bags proactively spilled: 0
Total records proactively spilled: 0

Job DAG:
job_1792936583670_0070  ->      job_1792936583670_0071,
job_1792936583670_0071  ->      job_1792936583670_0072,
job_1792936583670_0072


20:13:32 [INFO] MapReduceLauncher - Success!
(big,2)
(data,2)
(is,2)
(powerful,1)
(useful,1)
20:13:32 [INFO] Main - Pig script completed in 1 minute, 12 seconds and 723 milliseconds (72723 ms)

[ screenshot 5 - take it, then press Enter ]
```



Handwritten Questions

1. What is Apache Pig? Explain the role of Pig Latin in processing large datasets .
2. Write a Pig Latin script to load a text file named input.txt and display its contents.
3. Develop a Pig Latin script to split the lines of a text file into individual words and count the total number of words present in the file.
4. Consider the input text Big Data is useful. Big Data is powerful. Write the expected output of a word-count script that counts each distinct word separately. State any assumptions made regarding punctuation and case sensitivity. Acceptance Criteria

1 The student successfully loads the input text file using the appropriate Pig Latin command.
2 The student correctly tokenizes the input text into individual words using suitable string-processing operations.
3 The student executes the Pig Latin script to calculate the total word count or the frequency of each distinct word, as required.
4 The student verifies the output against the input data and explains the correctness of the word-count results.

① What is Apache Pig? Explain the role of pig latin in processing large datasets?

→ Roles of Pig Latin
- Processes large datasets efficiently.
- Supports filtering, grouping, sorting, joining.
- Converts scripts into MapReduce jobs.
- Reduces the complexity of writing Java MapReduce programs.

Apache Pig is a high level data processing platform used to analyse large datasets in Hadoop. It uses a scripting language called Pig Latin to simplify data processing.

② Write a Pig Latin script to load input.txt & display its contents.

=> data = LOAD 'input.txt' AS (line: chararray);
DUMP data;

Explaination :-
- LOAD reads text file
- AS specifies the schema
- DUMP display the contents on the screen.

③ Write a Pig Latin script to split lines into words and count the total number of words.

⇒ data = LOAD 'input.txt' AS (line : chararray);
words = FOREach data GENERATE FLATTEN
(tokenize (line));
grouped = Group word ALL;
result = FOR EACH grouped GENERATE COUNT (words);
DUMP result;

Explanation
- LOAD reads the file.
- TOKENIZE () splits lines into words
- FLATTEN () converts words into individual records.
- GROUP ALL groups all record together.
- DUMP displays the result.

4. Given the input text, write, the expected word-count output.
⇒ Big Data is useful. Big data is powerful.
Expected Output:

Word	Count
Big	2
Data	2
Is	2
useful	1
powerful	1

Total no: of words is 8.

Conclusion:A word-count application was implemented in Apache Pig. The text file was loaded from HDFS, split into words using TOKENIZE and FLATTEN, and counted using GROUP and COUNT. The script ran as MapReduce jobs and found 52 words, 37 distinct words and 5 lines, which matched the Linux verification. Thus, Pig performs word counting with far less code than Java MapReduce.